PLANEJAMENTO LÓGICO — TASKTRACKER

Bootcamp II — Desafio Cumulativo: Entrega Inicial (Fase 1)

Matheus Benjamim de Souza Bosco

Google Docs:	https://docs.google.com/document/d/1nII_QjO2q4gnBjNDh_nLL4e9npVl3URda9b-rx3Ecss/edit?usp=sharing
Vídeo (YouTube):	https://youtu.be/40QYAmNr-9Q

1. Apresentação e Entendimento do Problema

1.1 Identificação

Nome completo: Matheus Benjamim de Souza Bosco

Matrícula: 22612082

Curso: Engenharia de Software

Polo / Turma: UniCEUB — Noturno

E-mail institucional: matheus.bosco@sempreceub.com

1.2 O Problema a Ser Resolvido

No dia a dia, é comum que uma pessoa ou uma pequena equipe precise lidar com diversas tarefas simultaneamente — estudos, trabalho, projetos pessoais — e acabe perdendo o controle sobre o que já foi feito, o que está em andamento e o que ainda precisa ser feito. Sem um sistema de apoio, as tarefas costumam ser anotadas de forma dispersa (papéis, aplicativos de mensagem, memória), o que gera esquecimentos, retrabalho, atrasos e dificuldade em priorizar o que realmente importa. A ausência de um registro único e organizado impede que o usuário tenha uma visão clara de suas prioridades e prazos.

1.3 A Solução Proposta (TaskTracker)

O TaskTracker é um sistema de gerenciamento de tarefas organizado em duas telas complementares. A primeira é um quadro Kanban com três colunas — "A Fazer", "Fazendo" e "Feito" —, onde cada tarefa é representada por um card colorido de acordo com sua criticidade (verde para prioridade Baixa, amarelo para Média e vermelho para Alta), permitindo identificar visualmente o que exige mais atenção. Para manter o quadro limpo, os cards que chegam à coluna "Feito" são removidos automaticamente depois de um número de dias definido pelo próprio usuário, eliminando a necessidade de exclusão manual. A segunda tela é um calendário, posicionado logo abaixo do quadro Kanban, que exibe as mesmas tarefas organizadas por data limite, dando ao usuário um horizonte temporal claro do que está por vencer.

Essa é a visão completa do produto. Como o projeto é cumulativo, a Fase 1 (este documento) planeja essa lógica por escrito, e a Fase 2 implementa as mesmas regras de negócio em Python por meio de uma interface de terminal (sem renderização gráfica ainda) — o quadro Kanban e o calendário como telas visuais ficam definidos como evolução planejada para as fases seguintes, quando uma interface gráfica ou web passa a fazer sentido. As

tarefas são persistidas em um arquivo local (tasks.json), garantindo que nada se perca entre uma execução e outra do programa.

2. Decomposição do Problema e Dados do Sistema

2.1 Entradas de Dados

Informações que o usuário precisará digitar no sistema:

- **Título da tarefa:** nome curto e obrigatório que identifica a tarefa (ex.: "Estudar lógica de programação").
- **Descrição da tarefa:** detalhamento opcional sobre o que deve ser feito.
- **Prioridade:** nível de criticidade da tarefa, podendo ser Alta, Média ou Baixa — define também a cor do card (vermelho, amarelo ou verde).
- **Data limite:** prazo final para a conclusão da tarefa, usado também para posicionar a tarefa no calendário.
- **Valor de X (dias):** número de dias que o usuário define para que um card permaneça na coluna "Feito" antes de ser removido automaticamente.

2.2 Saídas de Dados

Informações que o sistema deverá exibir para o usuário:

- **Quadro Kanban:** as tarefas organizadas em três colunas — "A Fazer", "Fazendo" e "Feito" — exibidas como cards coloridos conforme a prioridade (verde, amarelo ou vermelho), cada um com título e data limite.
- **Calendário:** as mesmas tarefas organizadas por data limite, exibidas em um horizonte temporal (visão por dia/semana/mês) logo abaixo do quadro Kanban.
- **Mensagens de confirmação:** avisos de sucesso ao cadastrar uma tarefa ou mover um card entre colunas.
- **Mensagens de erro:** avisos quando o usuário tentar cadastrar uma tarefa inválida ou mover uma tarefa inexistente.
- **Total de tarefas por coluna:** contagem de quantas tarefas estão em "A Fazer", "Fazendo" e "Feito".

2.3 Regras do Sistema

Regras que o sistema deve obrigatoriamente respeitar:

- O sistema não pode aceitar o cadastro de uma tarefa sem título.
- A prioridade informada deve ser apenas "Alta", "Média" ou "Baixa" — nenhum outro valor é aceito, e cada uma corresponde a uma cor fixa (vermelho, amarelo, verde).
- Toda tarefa cadastrada deve nascer na coluna "A Fazer", só avançando para "Fazendo" ou "Feito" por ação explícita do usuário.
- Uma tarefa só pode estar em uma coluna por vez ("A Fazer", "Fazendo" ou "Feito").
- Ao mover uma tarefa para "Feito", o sistema registra a data em que ela entrou nessa coluna — essa data é a base para a remoção automática.

- Sempre que o sistema for aberto, ele verifica os cards da coluna "Feito" e remove automaticamente aqueles cuja permanência já ultrapassou o valor de X dias definido pelo usuário.

3. Receita Lógica do Sistema (Algoritmo em Português)

A seguir, o passo a passo lógico de cada funcionalidade principal do TaskTracker, descrito como se estivesse sendo explicado a uma pessoa leiga.

3.1 Passo a Passo do Cadastro de Tarefa

1. Exibir o menu principal e aguardar o usuário escolher a opção "Cadastrar nova tarefa".
2. Pedir ao usuário o título da tarefa.
3. Verificar se o título não está vazio; se estiver, exibir mensagem de erro e solicitar novamente.
4. Pedir ao usuário a descrição da tarefa (opcional).
5. Pedir ao usuário a prioridade da tarefa (Alta, Média ou Baixa).
6. Verificar se a prioridade informada é válida; se não for, solicitar novamente.
7. Pedir ao usuário a data limite da tarefa.
8. Salvar a tarefa em memória e persistir no arquivo tasks.json na coluna "A Fazer", com a cor definida pela prioridade.
9. Exibir mensagem de sucesso confirmando o cadastro.

3.2 Passo a Passo da Visualização do Quadro Kanban

1. Antes de exibir, executar a rotina de expiração automática (seção 3.4), removendo cards vencidos da coluna "Feito".
2. Acessar a lista de tarefas, carregando-a do arquivo tasks.json.
3. Verificar se existe alguma tarefa cadastrada.
4. Se a lista estiver vazia, avisar o usuário de que não há tarefas cadastradas.
5. Se houver tarefas, agrupá-las nas três colunas: "A Fazer", "Fazendo" e "Feito".
6. Exibir cada card com título, data limite e cor correspondente à prioridade (verde, amarelo ou vermelho).
7. Ao final, exibir o total de tarefas em cada uma das três colunas.

3.3 Passo a Passo da Movimentação de Tarefa entre Colunas

1. Pedir ao usuário qual tarefa deseja mover e para qual coluna ("A Fazer", "Fazendo" ou "Feito").
2. Buscar a tarefa correspondente na lista de tarefas.
3. Verificar se a tarefa existe; se não existir, exibir mensagem de erro.
4. Verificar se a coluna de destino informada é válida; se não for, solicitar novamente.
5. Atualizar a coluna da tarefa para o destino informado.
6. Se o destino for "Feito", registrar a data atual como "data de conclusão" da tarefa.
7. Salvar a alteração no tasks.json e exibir mensagem de sucesso.

3.4 Passo a Passo da Expiração Automática de Cards Concluídos

1. Ao iniciar o sistema, percorrer todas as tarefas que estão na coluna "Feito".

2. Para cada uma, calcular a diferença, em dias, entre a data atual e a "data de conclusão" registrada.
3. Comparar essa diferença com o valor de X (dias) definido pelo usuário nas configurações.
4. Se a diferença for maior ou igual a X, remover a tarefa da lista e do arquivo tasks.json.
5. Se for menor que X, manter a tarefa normalmente no quadro.
6. Repetir o processo para todos os cards da coluna "Feito" antes de exibir o quadro ao usuário.

3.5 Passo a Passo da Visualização em Calendário

1. Acessar a lista de tarefas carregada do tasks.json (já sem os cards expirados).
2. Agrupar as tarefas pela data limite de cada uma.
3. Exibir um calendário (por padrão, com o mês atual em destaque) posicionado logo abaixo do quadro Kanban.
4. Em cada dia do calendário que tiver tarefa associada, exibir o título da tarefa com a cor correspondente à sua prioridade.
5. Permitir ao usuário navegar entre meses para visualizar tarefas passadas ou futuras.

4. Mapeamento Arquitetural para Git e GitHub (Preparação para a Fase 2)

4.1 Desenho da Árvore de Repositório

Em vez de concentrar toda a lógica em um único arquivo, o projeto será organizado em camadas, separando responsabilidades: modelo de dados, regras de negócio, persistência e interface com o usuário. Essa separação facilita testes, manutenção e futuras evoluções do sistema (como trocar o armazenamento em JSON por um banco de dados na Fase 3, sem precisar alterar as regras de negócio nem a interface).

- **models/:** define a estrutura de uma tarefa (Task): título, descrição, prioridade (e cor derivada), coluna (A Fazer/Fazendo/Feito), data limite e data de conclusão.
- **services/:** concentra as regras de negócio — cadastrar, mover tarefas entre colunas, validar dados e expirar cards antigos da coluna "Feito". É a camada que a Fase 3 vai testar automaticamente.
- **storage/:** responsável por ler e escrever o arquivo tasks.json. É a única camada que sabe como os dados são persistidos — se um dia trocarmos JSON por banco de dados, só esta camada muda.
- **utils/:** funções auxiliares de data, como calcular há quantos dias uma tarefa está na coluna "Feito" — usada pela regra de expiração automática.
- **cli/:** camada de interface atual: exibe o quadro Kanban e o calendário em formato texto no terminal e lê a escolha do usuário. É a camada que será substituída por uma interface gráfica (web) nas fases futuras, sem exigir mudanças nas demais.

```
tasktracker-projeto/
├── README.md                    # Apresentação do projeto no GitHub
├── .gitignore                   # Ignora arquivos temporários
├── requirements.txt             # Dependências do projeto Python
├── docs/
│   └── planejamento_logico.pdf  # Este planejamento da Fase 1
├── src/
│   ├── main.py                  # Ponto de entrada: inicia a aplicação
│   ├── models/
│   │   └── task.py              # Classe Task — estrutura de uma tarefa
│   ├── services/
│   │   ├── task_service.py      # Cadastrar, mover e validar tarefas
│   │   └── expiration_service.py # Expira cards antigos da coluna Feito
│   ├── storage/
│   │   └── json_storage.py      # Leitura e escrita do tasks.json
│   ├── utils/
│   │   └── date_utils.py        # Cálculo de dias entre datas
│   ├── cli/
│   │   └── menu.py              # Quadro Kanban e calendário em texto
│   └── data/
│       ├── tasks.json           # Arquivo de persistência das tarefas
│       └── config.json          # Guarda o valor de X (dias) definido
│                               #   pelo usuário
└── tests/
    ├── test_task_service.py     # Testes das regras de negócio
    ├── test_expiration.py       # Testes da expiração automática
    └── test_json_storage.py     # Testes da camada de persistência
```

4.2 Plano de Aprendizado para Git e GitHub

Na Fase 2 deste projeto, espero aprender a criar e configurar um repositório no GitHub para versionar o código do TaskTracker, entendendo na prática o que é um commit e como registrar alterações de forma organizada e descritiva ao longo do desenvolvimento. Também pretendo aprender a criar branches para desenvolver novas funcionalidades sem comprometer a versão principal do projeto, a resolver conflitos de código quando eles surgirem e a utilizar o histórico de versões para acompanhar a evolução do sistema. Por fim, espero compreender o fluxo completo de publicar um projeto no GitHub, desde o primeiro commit até a documentação final em um README claro e informativo.

5. Link da Apresentação em Vídeo

Link do vídeo de apresentação (gravado no YouTube, formato horizontal, público ou não listado):

https://youtu.be/40QYAmNr-9Q